

Strukture podataka i algoritmi

Nizovi i liste

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

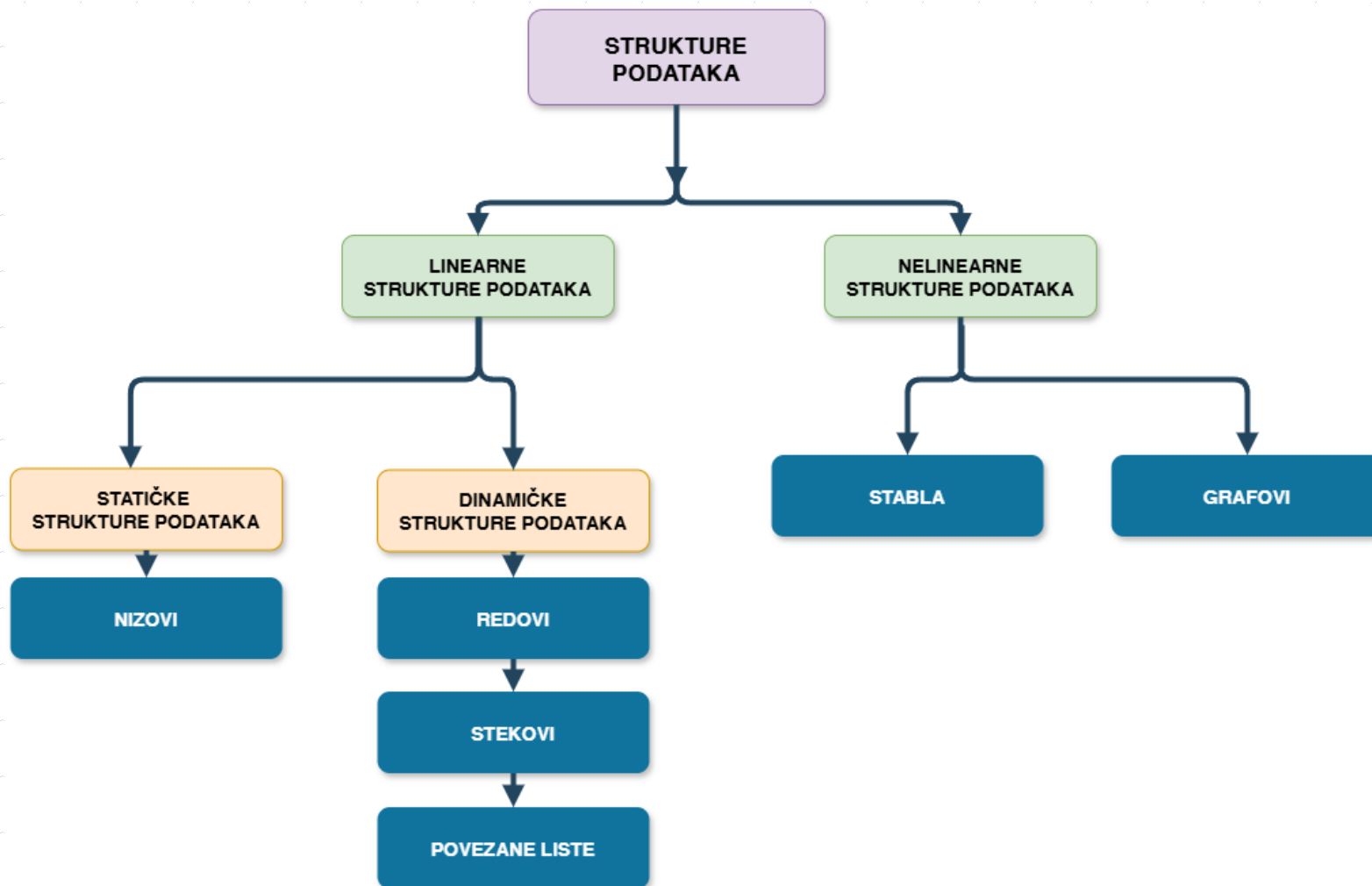
Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Klasifikacija struktura podataka
2. Strukture podataka: Nizovi
3. Strukture podataka: Liste

1. Klasifikacija struktura podataka

Klasifikacija struktura podataka



Strukture podataka

- Strukture podataka su složeni tipovi podataka koji nastaju primenom apstrakcije agregacije, tj. objedinjavanjem više jednostavnijih tipova podataka
- Svaka konkretna vrednost strukture podataka sastoji se od jednostavnijih delova, koji predstavljaju njene elemente koji mogu biti prosti ili složeni
- Elementi u strukturi podataka mogu biti u međusobnom odnosu (relaciji)
- Formalno, struktura podataka može biti definisana kao uređeni par

$$S = (E, p)$$

pri čemu je E skup elemenata, a p je binarna relacija kojom je uređen skup E

Strukture podataka - vrste

- Pojedine vrste struktura podataka se definišu na osnovu toga kakav je njihov odnos, tj. kakve su osobine relacije p
- Najvažnija osobina relacije p je njena **kardinalnost**, koliko elemenata odgovara jednom elementu u datoj strukturi i obratno
- Na osnovu kardinalnosti relacije p mogu se definisati sledeće vrste struktura:
 - **0:0** (ne postoji uređenje) – kolekcije i skupovi
 - **1:1** (linearno uređenje) – linearne strukture
 - **1:M** – stabla
 - **M:M** – grafovi i mreže

Statičke i dinamičke linearne strukture

- Sa aspekta implementacije, linearne strukture mogu biti:
 - a) **Statičke**
 - vektorske strukture (*array-based*)
 - b) **Dinamičke**
 - ulančane strukture (*linked-based*)
- Statičke strukture karakteriše statička alokacija memorije, što znači da se raspoloživa memorija određuje programskim kodom (loša inicijalna procena memorije dovodi do njenog nepotrebnog zauzeća ili nedostatka)
- Dinamičke strukture karakteriše dinamička alokacija memorije, što znači da se ona vrši po potrebi i u toku izvršavanja programa (rezerviše se tačno onoliko memorije koliko je potrebno, alocirani prostor se oslobađa kada dinamički podaci više nisu potrebni)

2. Strukture podataka: Nizovi

Niz kao struktura podataka

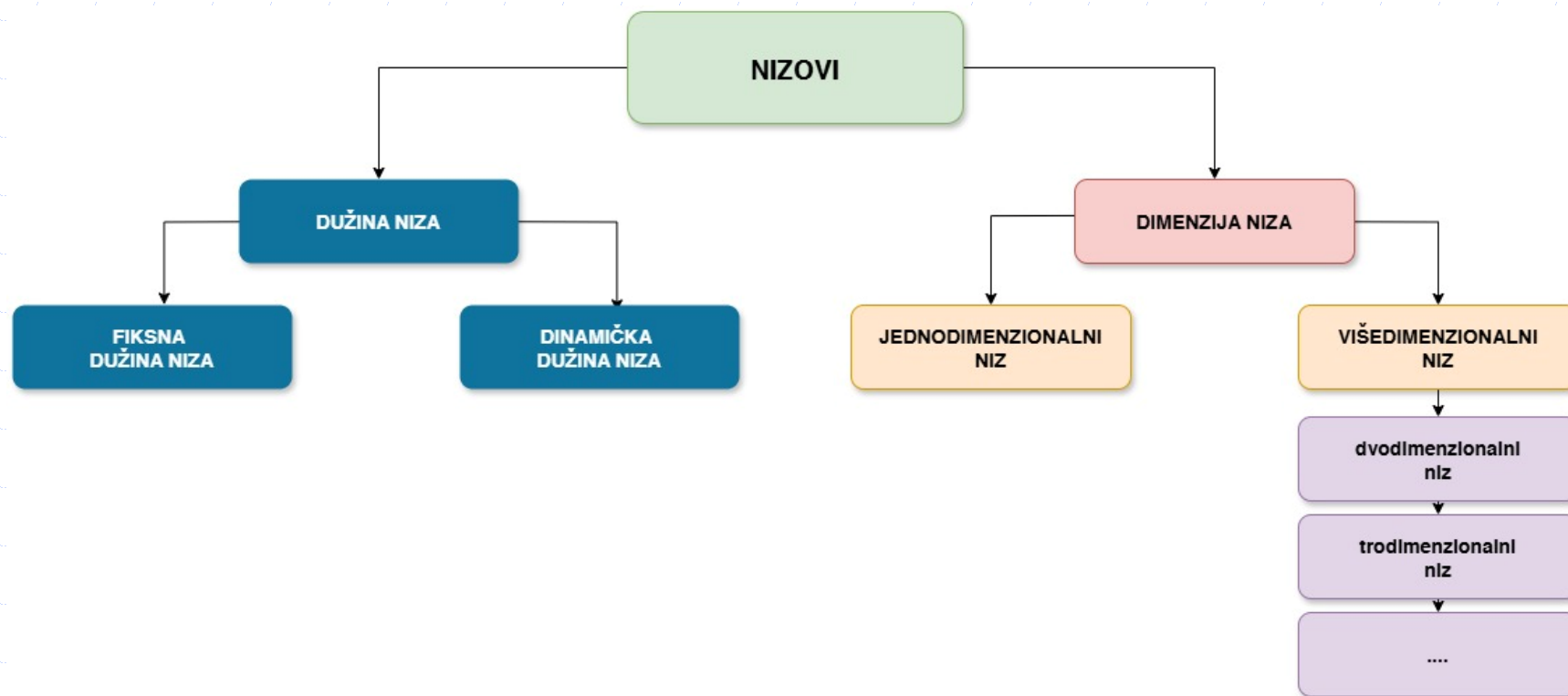
- **NIZ** (ARRAY) je struktura podataka koja se sastoji od članova (elemenata) smeštenih na susednim memorijskim lokacijama
- Računar niz „vidi“ kao memorijske lokacije sa jedinstvenim adresama, kojima procesor pomoću posebnih instrukcija pristupa kao članovima niza
- Nizovi su zbog toga pogodna struktura podataka za slučajeve kada treba brz pristup podacima pomoću numeričkog indeksa
- Osnovna terminologija nizova:
 - **Član niza**: Predstavlja memorijsku lokaciju niza kojoj se pristupa putem njegovog indeksa
 - **Indeks niza**: članu niza se pristupa preko njihovog **indeksa koji počinje od 0**
 - **Dužina niza**: Određena je brojem članova koje on sadrži

Niz kao struktura podataka

- Sekvencijalna uređenost niza znači da su njegovi elementi numerisani redom od prvog do poslednjeg
- Ukupan broj elemenata niza se naziva dužina ili veličina niza
- Dužina niza se mora unapred zadati
- Za nizove se definišu sledeće operacije:
 - **Indeksiranje**: pristup elementima putem celobrojnog indeksa
 - **Dodavanje**: dodavanje novog elementa na određeno mesto u nizu
 - **Brisanje**: brisanje indeksom određenog elementa niza
 - **Pretraživanje**: pronalaženje tražene vrednosti unutar niza
 - **Slicing**: pristup podnizu nekog niza

Nizovi - podela

- Osnovna podela nizova je prema njihovoj dužini (fiksni ili dinamički) i prema dimenziji (1-, 2- ili 3-dimenzionalni...)



Indeksiranje niza

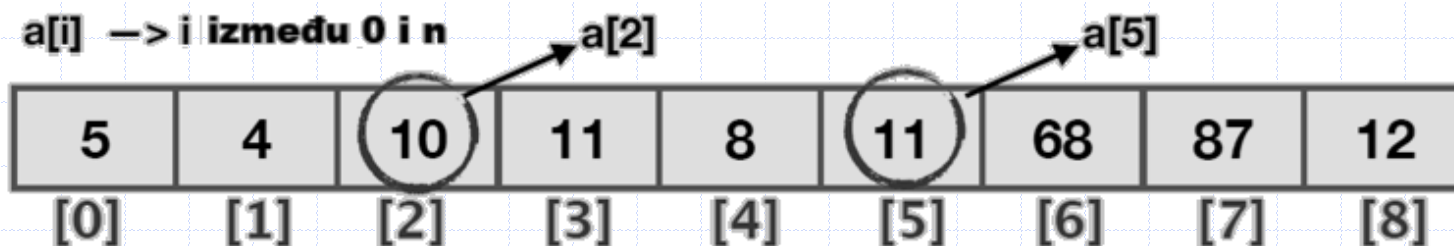
- Za svaki se niz mora znati adresa njegovog prvog elementa koja predstavlja početnu adresu niza
- Evidenciju o adresi vodi interpreter programskog jezika
- S obzirom da interpreter vodi računa o tome na kojoj adresi počinje svaki niz, indeksiranje niza je jednostavna operacija
- Da bi se odredila adresa indeksiranog elementa, početnoj adresi se doda indeks
- Ako **A** sadrži adresu prvog člana niza, tada se element niza na indeksu **i** računa se po formuli

$$A[i] = A + iB$$

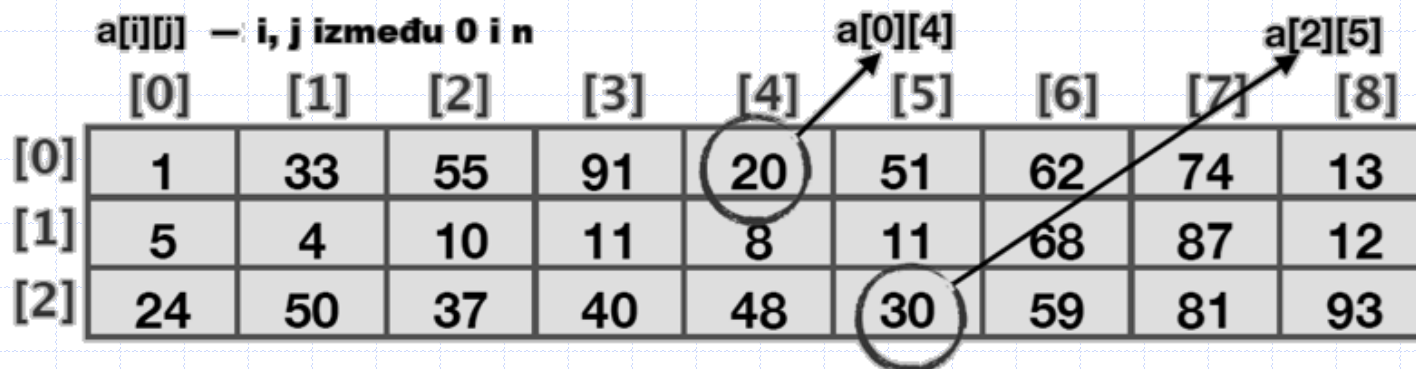
pri čemu **B** predstavlja veličinu elementa niza datu u bajtovima, i uz pretpostavku da je niz homogen, tj. da su svi elementi niza istog tipa i da se prvi element nalazi na **indeksu 0**

Tipovi nizova

- **Jednodimenzionalni niz:** elementi niza se deklarišu jednim indeksom **i** koji se definiše od pozicije [0] do [n]

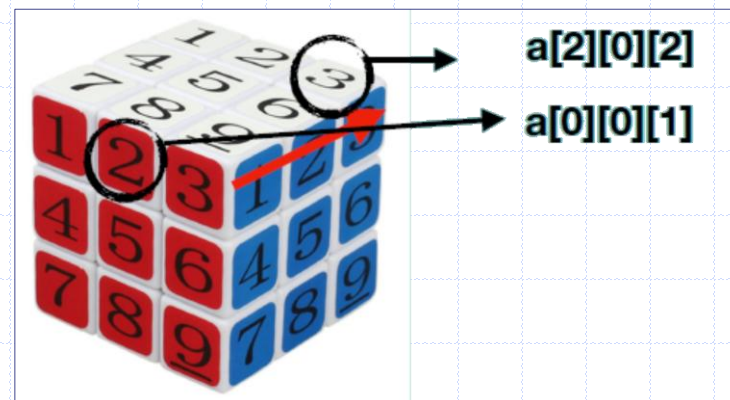
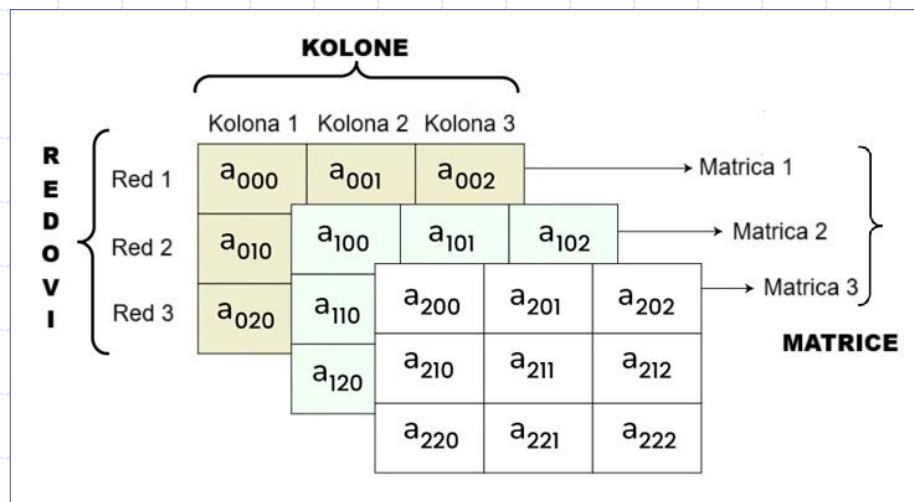


- **Dvodimenzionalni niz:** elementi niza se deklarišu sa dva indeksa **i, j** koji su reference redova tj. kolona



Tipovi nizova

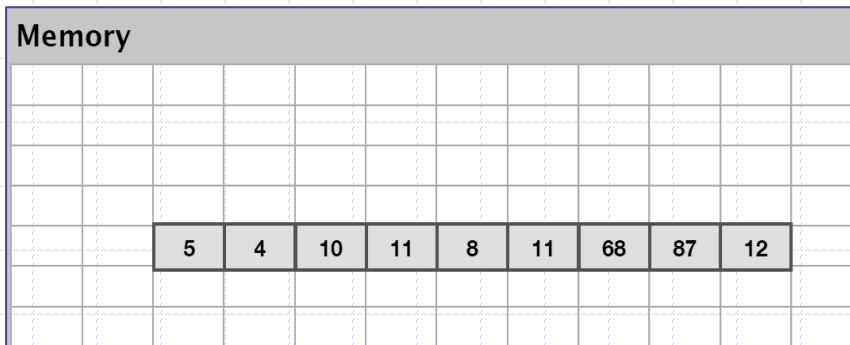
- **Trodimenzijski niz:** elementi niza se deklariraju sa tri indeksa npr. **$a[i][j][k]$** koji se definišu od pozicije [0] do [n]



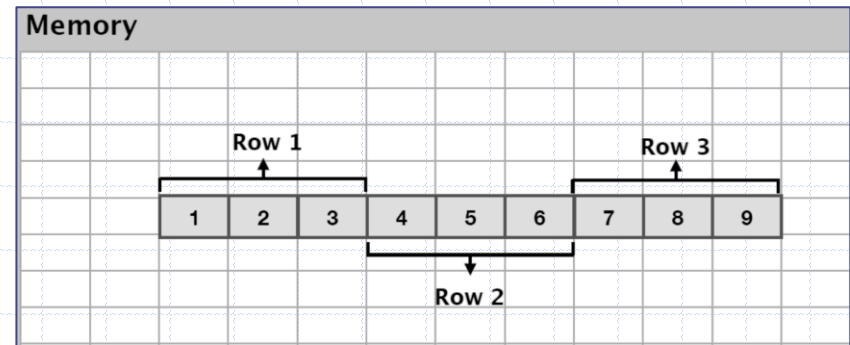
- Pošto se elementi niza/reference čuvaju na susednim lokacijama u memoriji, nizovi imaju prednost lokalnosti reference
- Nizovi nisu pogodni za operacije umetanja elemenata u sredinu, brisanja iz sredine i pretraživanja u nesortiranim podacima
- Nizovi su osnova drugih struktura: stekova, redova, grafova, heš tabela

Memorisanje nizova

- Memorijski prostor za smeštanje niza je kontinualan, tako da se elementi bilo kog tipa niza smeštaju u **susednim memorijskim lokacijama**

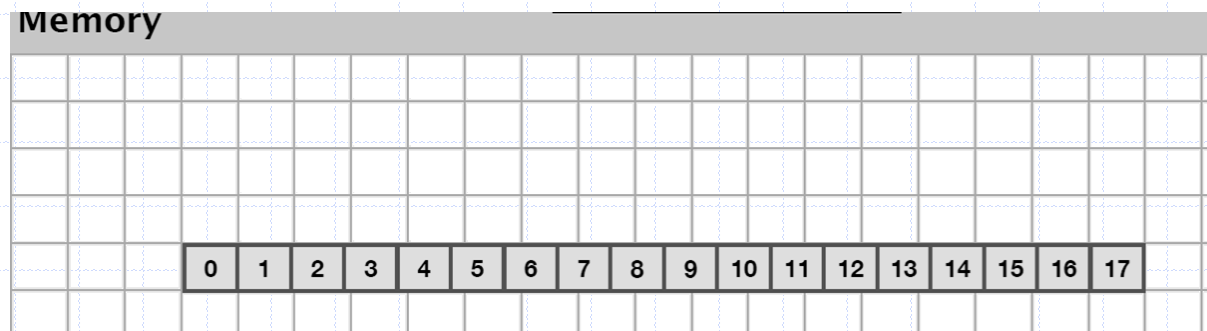


Jednodimenzionalni niz u memoriji



Dvodimenzionalni niz u memoriji

```
[[[ 0, 1, 2],  
 [ 3, 4, 5]],  
  
[[ 6, 7, 8],  
 [ 9, 10, 11]],  
  
[[12, 13, 14],  
 [15, 16, 17]]]
```



Trodimenzionalni niz u memoriji

Memorijska forma niza

- Svaki element niza se čuva u kontinualnim memorijskim lokacijama
- Kada se niz inicijalizuje, njegovi elementi se alociraju sekvencijalno (jedan za drugim) u memoriji, što omogućava efikasan pristup i manipulacije elementima niza
- Elementi niza su reference na objekte, što znači da svaki element u nizu ukazuje na memorijsku lokaciju gde je objekat smešten
- Operacija indeksiranja niza (psudokod):

Parametri: A - niz s elementima tipa T ; i - indeks niza

Rezultat: adresa elementa na indeksu i

```
1: function INDEKS( $A : T[]$ ,  $i : INT$ )  
2:   return  $A + i * VELIČINA(T)$   
3: end function
```

- Funkcija **INDEKS** vraća adresu elementa na indeksu i dok se veličina elementa dobija se na osnovu veličine elementa i adrese
- Indeksiranje niza je vrlo efikasna operacija, izvršava se u const. vremenu i ne zavisi od veličine niza, što znači da indeksiranje niza od 5 miliona elemenata ne zahteva više vremena od indeksiranja niza od 5 elemenata

Tipovi nizova prema veličini

- Dve osnovne podele nizova prema njihovoj veličini:
 - a) Fiksni nizovi
 - b) Dinamički nizovi
- Kod fiksnih nizova ne možemo menjati njihovu veličinu, u uglastim zagradama `[]` navodi se fiksna veličine memorije. Za ove nizove statička alokacija memorije nije poželjna jer ne znamo unapred veličinu niza i može doći do gubitka podataka

```
# Kreiranje niza fiksne dužine 5, inicijalizovan sa nulama  
arr = [0] * 5  
# Output za niz fiksne dužine  
print(arr)
```



`[0, 0, 0, 0, 0]`

- Kod dinamičkih nizova veličina niza se menja prema zahtevima tokom izvršavanja kôda, pa se ne mora voditi računa o veličini niza, moguće je dodavanje i uklanjanje elemenata niza po potrebi, a memorija se u ovim nizovima dinamički dodeljuje i dealocira. Takvi nizovi, nakon što se popune, postojeće elemente kopiraju u novi, veći deo memorije. U Pythonu se za ovakve nizove koriste LISTE

```
# Dinamički niz  
arr = []
```

Nizovi u Python jeziku

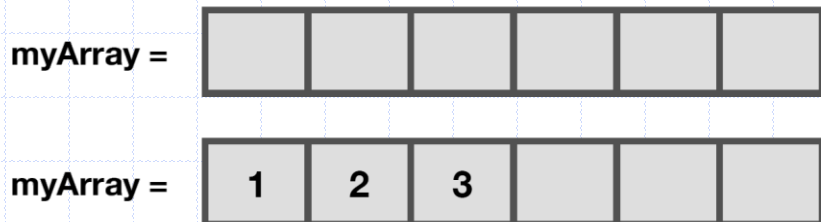
- **Indeksiranje:** U Pythonu se elementima unutar niza pristupa pomoću **indeksa** koji počinje od **nule (0)**, što olakšava pristup ili ažuriranje određenog elementa unutar niza
- **Upotreba memorije:** Nizovi koriste manje memorije od listi, jer koriste kontinualne memorijske lokacije sa fiksnom veličinom identičnih elemenata
- **Susedne memorijske lokacije:** Elementi nizova su smešteni u susednim blokovima memorije, što omogućava izvršavanje operacije čitanja/pisanja brže od bilo kojih drugih „razbacanih“ memorijskih struktura
- **Kompatibilnost biblioteka:** Nizovi su kompatibilni sa bibliotekama poput **Numpy**, što značajno proširuje njihovu funkcionalnost

Operacije za rad sa nizovima

Osnovne operacije za rad sa nizovima su:

- 1) Kreiranje niza
- 2) Dodavanje novog elementa niza (*insert*)
 - Dodavanje elementa na početak niza (indeks 0)
 - Dodavanje elementa na određeni lokaciju u nizu
 - Dodavanje elementa na kraj niza
- 3) Obilazak (*traversal*) niza
- 4) Pretraživanje po vrednosti elementa niza
- 5) Brisanje vrednosti elementa niza
 - Brisanje elementa na početku niza (indeks 0)
 - Brisanje elementa na određenoj lokaciji u nizu
 - Brisanje elementa na kraju niza

Deklaracija i inicijalizacija 1D niza



- Deklaracija niza: Nizovi se mogu deklarirati na različite načine u različitim jezicima. U Python jeziku se za deklaraciju niza koriste **liste**

```
# U Pythonu se sve liste kreiraju na isti način  
myArray = []
```

- Inicijalizacija niza: Nizovi se inicijalizuju na različite načine

```
# Lista sa integer tipom elemenata  
myArray = [1, 2, 3, 4, 5]  
# Lista sa string tipom elemenata  
myArray = ['a', 'b', 'c', 'd', 'e']  
# Lista sa float tipom elemenata  
myArray = [1.4, 2.0, 24.0, 5.0, 0.0] # Sve float vrednosti
```

Nizovi – `array` modul

- U Python jeziku, nizovi su vrsta `kontejnera` koji može efikasnije da skladišti elemente istog tipa podataka
- Koristi se ugrađeni (built-in) modul `array` za rad sa velikim numeričkim podacima gde su performanse i efikasnost memorije bitni parametri
- Nizovi su idealni za matematičke ili naučne proračune kod kojih je konzistentnost tipa podataka ključna
- Upotreba modula `array` u Pytonu ima sledeću sintaksu:

`array(typecode, [value1, value2...])`

`typecode` – znak koji predstavlja tip podatka (i za integer, f za float...)

`value1, value2` – iterabilna lista vrednosti koje odgovaraju typecode-u

- Funkcija `array()` kreira niz sa elementima homogenog tipa podataka i koristi za rad sa numeričkim nizovima fiksnog tipa

Nizovi – array modul

Append() Method:

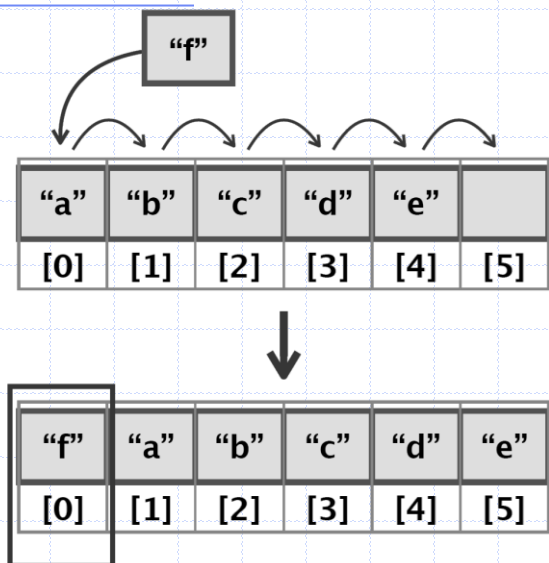
append() metoda dodaje određenu vrednost na kraj niza

```
import array
# kreiranje niza celih brojeva
arr = array.array('i', [1, 2, 3])
# pristup prvom elementu niza
print(arr[0])
# dodavanje elementa u niz
arr.append(5)
print(arr)
```

```
1
array('i', [1, 2, 3, 5])
```

Type Code	C Type	Python Type	Minimum size in Bytes
'b'	signed char	int	1
'B'	unsigned char	int	1
'u'	Py_UNICODE	Unicode character	2
'h'	signed short	int	2
'H'	unsigned short	int	2
'i'	signed int	int	2
'I'	unsigned int	int	2
'l'	signed long	int	4
'L'	unsigned long	int	4
'q'	signed long long	int	8
'Q'	unsigned long long	int	8
'f'	float	float	4
'd'	double	float	8

Dodavanje novog elementa 1D niza



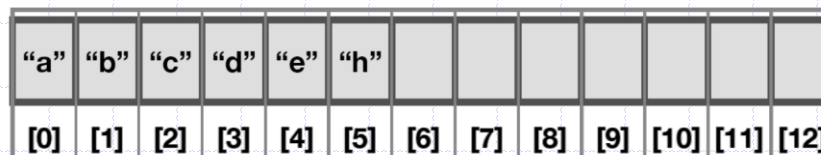
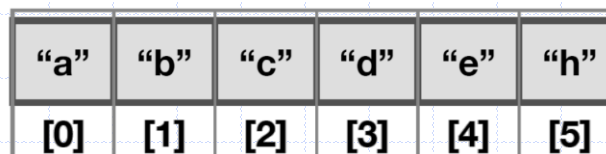
Dodavanje novog elementa niza na početak

```
# Insert elementa niza na početak-ugrađene metoda insert
arr = [10, 20, 30, 40]
element = 50
print("Array before insertion")
for i in range(len(arr)):
    print(arr[i], end=" ")
# Insert elementa na pocetak
arr.insert(0, element)

print("\nArray after insertion")
for i in range(len(arr)):
    print(arr[i], end=" ")
```

Output

```
Array before insertion
10 20 30 40
Array after insertion
50 10 20 30 40
```



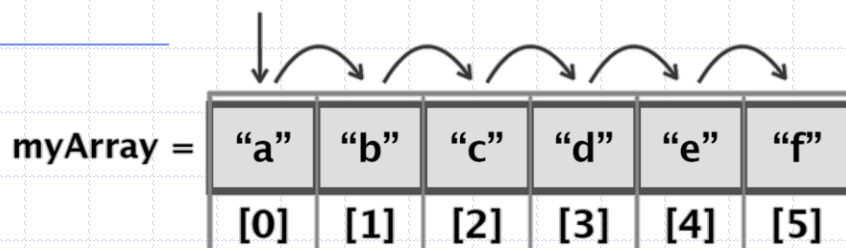
Dodavanje novog elementa za popunjen niz

```
# Insert elementa niza na početak-custom metoda
arr = [10, 20, 30, 40]
n=4
element = 50
print("Array before insertion")
for i in range(len(arr)):
    print(arr[i], end=" ")
# Pomeriti sve elemente niza u desno
for i in range(n - 1, -1, -1):
    arr[i + 1] = arr[i]
# Insert elementa na pocetak
arr[0]=element
print("\nArray after insertion")
for i in range(n+1):
    print(arr[i], end=" ")
```

Output

```
Array before insertion
10 20 30 40
Array after insertion
50 10 20 30 40
```

Obilazak 1D niza



```
def traverseArray(array):  
    for i in array:-----> O(n)  
        print(i)-----> O(1) }----> O(n)
```

myArray[0] = "a"
myArray[1] = "b"
myArray[2] = "c"
myArray[3] = "d"
myArray[4] = "e"
myArray[5] = "f"

- Obilazak niza je postupak sekvencijalnog pristupa svakom njegovom elementu, najčešće zbog pretraživanja, sortiranja ili izmene elemenata
- Najčešće metode uključuju iteraciju kroz niz korišćenjem petlji **for** ili **while**
- Obilazak niza: linearni i obrnuti

```
arr = [10, 20, 30, 40, 50]  
print("Traversal using for loop: ",  
end=' ')  
for i in arr:  
    print(i, end=' ')  
print()
```

Obilazak niza FOR petljom

```
arr = [10, 20, 30, 40, 50]  
n = len(arr)  
i = 0  
print("Traversal using while loop: ",  
end=" ")  
while i < n:  
    print(arr[i], end=" ")  
    i += 1  
print()
```

Obilazak niza WHILE petljom

Pretraživanje niza

- Najjednostavnije pretraživanje niza je **linearno** kod kojeg se pretpostavlja da elementi niza nisu uređeni, pa se ne zna gde se u nizu nalazi traženi element
- Efikasnije pretraživanje niza se postiže sortiranjem njegovih elemenata po veličini, i sortiranje se vrši ili za brojeve, ili bilo koji tip vrednosti koji se može porediti
- Kod pretraživanja sortiranog niza, najveće poboljšanje je prekid pretraživanja čim se dođe do prvog elementa u nizu koji je npr. veći od onog koji tražimo
- Ako su elementi niza sortirani, najefikasnija tehnika pretraživanja je binarno pretraživanje

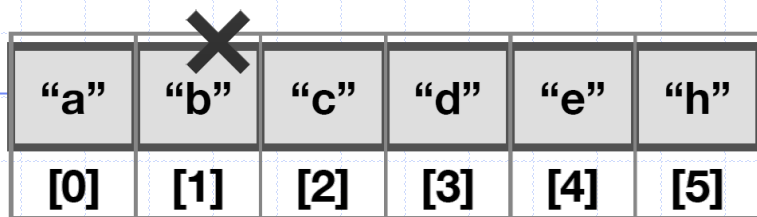
```
def searchInArray(array, value):  
    for i in array: .....→ O(n)  
        if i == value: .....→ O(1)  
            return arr.index(value).....→ O(n)  
    return "The element does not exist in this array" .....→ O(1)
```

} → $O(n^2)$

```
def searchInArray(array, value):  
    for i in array: .....→ O(n)  
        if i == value: .....→ O(1)  
            return True .....→ O(1)  
    return "The element does not exist in this array".....→ O(1)
```

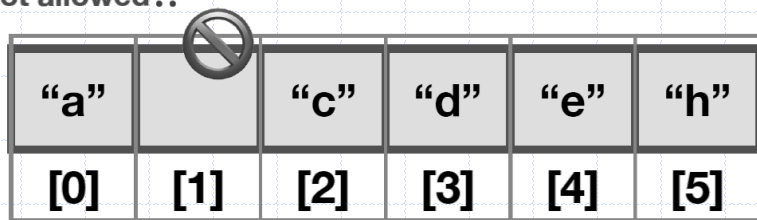
} $O(n)$

Brisanje elemenata niza

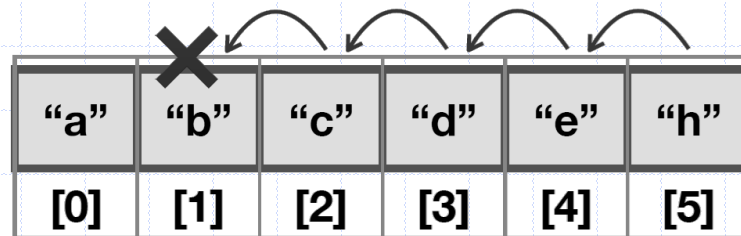


"a"	"b"	"c"	"d"	"e"	"h"
[0]	[1]	[2]	[3]	[4]	[5]

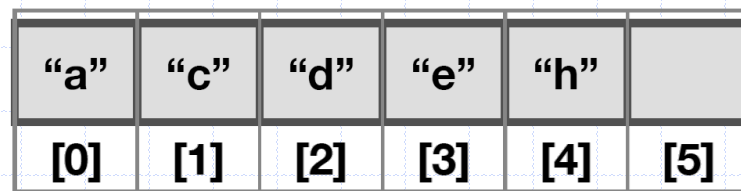
Not allowed !!



"a"		"c"	"d"	"e"	"h"
[0]	[1]	[2]	[3]	[4]	[5]



"a"	"b"	"c"	"d"	"e"	"h"
[0]	[1]	[2]	[3]	[4]	[5]



"a"	"c"	"d"	"e"	"h"	
[0]	[1]	[2]	[3]	[4]	[5]

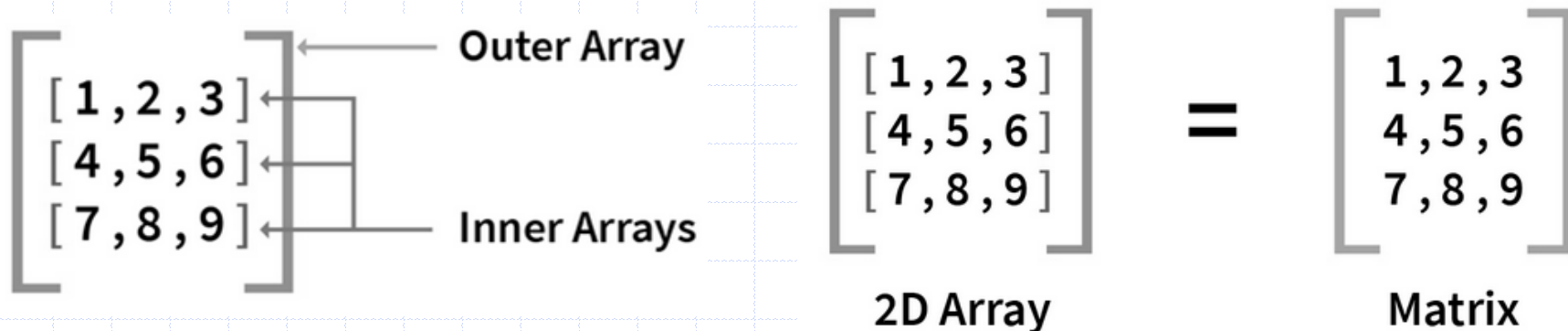
- **Brisanje elemenata niza:** Podrazumeva postupak uklanjanja elementa sa određene pozicije uz očuvanje redosleda preostalih elemenata. Za razliku od lista gde je brisanje efikasno, uklanjanje elementa iz niza zahteva pomeranje (*shift*) elemenata da bi se popunila lokacija izbrisanog elementa
- Pošto nizovi imaju kontinualnu alokaciju memorije, brisanje elementa ne smanjuje veličinu dodeljene memorije, pa se koristi:
 - Identifikacija pozicije: Pronalaženje indeksa elementa koji treba izbrisati
 - Pomeranje elemenata: Pomeranje elemenata posle izbrisanog elementa jednu poziciju ulevo
 - Ažuriranje veličine (ako je primenljivo): Ako se koristi dinamički niz, veličina može biti smanjena

Vremenska i prostorna složenost operacija kod 1D nizova

	Vremenska složenost	Prostorna složenost
Kreiranje niza	$O(1)$	$O(n)$
Unos novog elementa niza	$O(1)/O(n)$	$O(1)$
Obilazak niza	$O(n)$	$O(1)$
Pretraživanje niza	$O(n)$	$O(1)$
Brisanje elementa niza	$O(1)/O(n)$	$O(1)$

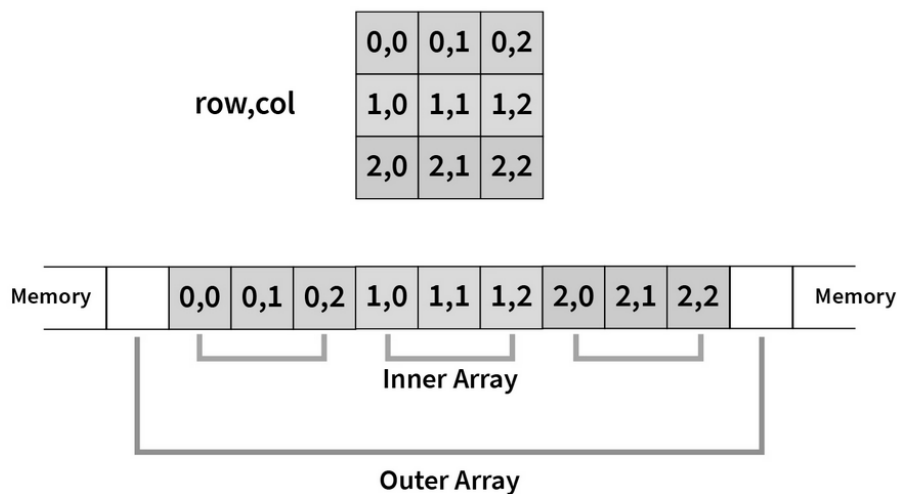
2D nizovi (matrice)

- U dvodimenzionalnim nizovima elementi niza se deklarišu sa dva indeksa **i, j** koji su reference redova tj. kolona **MATRICE**
- Matrica elemenata predstavlja konceptijsku tabelu elemenata koji su organizovani u redove i kolone i za koju se unapred moraju znati brojevi redova i kolona, što predstavlja veličinu matrice
- Veličina matrice se ne može menjati, što proizilazi iz nepromenljivosti veličine jednodimenzionalnog niza
- Ukoliko elementi 1D niza predstavljaju matrice, dobija se 3D niz ili niz matrica
- **Interni nizovi** (*inner arrays*) čine 1D nizove redova, dok **spoljašnji niz** (*outer array*) predstavlja celu formu matrice



2D nizovi (matrice)

- 2D niz u Python jeziku je dvodimenzionalna struktura podataka koja se kontinualno čuva u memoriji što znači da su elementi linearno raspoređeni i da je svaki element povezan sa svojim prethodnim i sledećim elementom



	0	1	2
0	[1, 2, 3]		
1	[4, 5, 6]		
2	[7, 8, 9]		

2D Array

- Kontinualni raspored elemenata u memoriji čini nizove nasumično dostupnim, pa je vreme pristupa bilo kom elementu niza isto i nezavisno od veličine niza
- 2D nizovi u Pythonu su indeksiraju od nule, prvi indeks je 0
- Kreiranje 2D niza u Pythonu:

ime_niza=[broj_redova][broj_kolona]

Kreiranje 2D niza (matrice)

- Mogući različiti načini:

```
# Kreiranje 2D niza 3x4,  
# svi elementi su nule  
rows = 3  
cols = 4  
arr = [[0 for j in range(cols)]  
        for i in range(rows)]  
  
print(arr)
```

```
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

Output

```
[1, 0, 0, 0, 0]  
[1, 0, 0, 0, 0]  
[1, 0, 0, 0, 0]  
[1, 0, 0, 0, 0]  
[1, 0, 0, 0, 0]  
[1, 0, 0, 0, 0]  
[0, 0, 0, 0, 0]  
[0, 0, 0, 0, 0]  
[0, 0, 0, 0, 0]  
[0, 0, 0, 0, 0]
```



```
rows, cols = (5, 5)  
arr = [[0]*cols]*rows  
print(arr)
```

```
[[0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
```

```
# Inicijalizacija 2D niza  
rows, cols = (5, 5)  
  
arr = [[0]*cols]*rows  
# Promena prvog elementa prvog reda  
# u vrednost 1 i print niza  
arr[0][0] = 1  
  
for row in arr:  
    print(row)
```

Vremenska i prostorna složenost operacija kod 2D nizova

	Vremenska složenost	Prostorna složenost
Kreiranje 2D niza	$O(1)$	$O(mn^*)$
Unos novog reda/kolone 2D niza	$O(mn)/O(1)$	$O(1)$
Obilazak 2D niza	$O(mn)$	$O(1)$
Pretraživanje 2D niza	$O(mn)$	$O(1)$
Brisanje reda/kolone 2D niza	$O(mn)/O(1)$	$O(1)$

* m – broj redova matrice
n – broj kolona matrice

2. Struktura podataka: Liste

Liste

- Ako je efikasno dodavanje i uklanjanje elemenata važnije od indeksiranja, tada razmatramo upotrebu **povezanih struktura podataka** (*linked data structures*)
- Kod takvih struktura podataka elementi nisu smešteni u memoriji jedan iza drugog, niti je jedan deo memorije unapred rezervisan za određeni broj elemenata, već se memorija rezerviše za svaki element posebno i svaki od tih elemenata obuhvata jednu ili više adresa na kojima se nalaze elementi te strukture koji su sa njim povezani
- **Povezana lista** (*linked list*) kao struktura podataka se tako naziva jer predstavlja jedan niz elemenata koji su organizovani na nešto drugačiji način od klasčnog niza, kako bi se omogućilo efikasno dodavanje i brisanje elemenata
- Liste (povezane liste) su strukture podataka slične nizovima, jer predstavljaju sekvence elemenata istog tipa

Liste

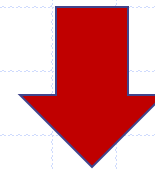
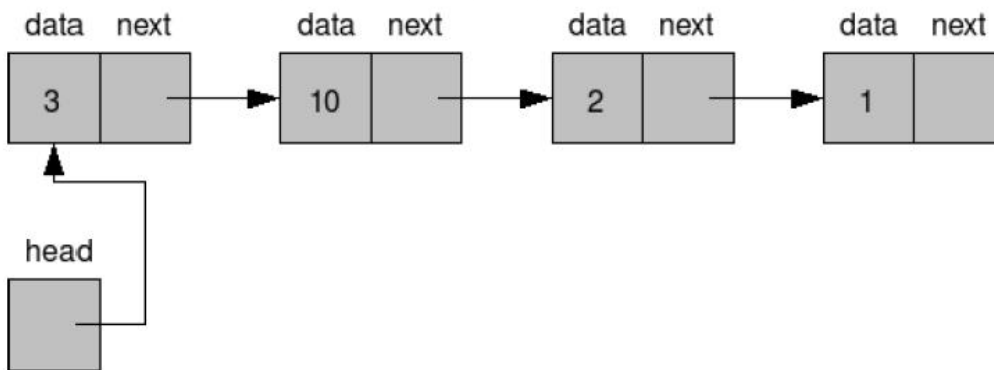
- Liste se upotrebljavaju za čuvanje podataka (slično kao i nizovi)
- Prednosti liste u odnosu na statički/dinamički niz: efikasno se dodaju i brišu elementi (konstantna složenost)
- Mane liste u odnosu na statički/dinamički niz: pristup n-tom elementu liste je linearne složenosti
- Liste koristimo ukoliko je potrebno da:
 - često pristupamo podacima i menjamo njihovu vrednost
 - često dodajemo nove podatke na kraj strukture
 - često dodajemo nove podatke na početak strukture
 - često dodajemo nove podatke na odgovarajuće mesto (tj. bilo gde)
 - često dodajemo i brišemo podatke
- Najbolji izbor strukture podataka (niz statički/dinamički ili lista) je vezan za specifičnosti konkretnog problema i najvažnije zahteve

Nizovi vs. Liste

LISTE	NIZOVI
Elementi liste mogu biti različitog tipa	Elementi niza su uvek istog tipa
Nije potreban eksplicitni <code>import</code> modula za deklaraciju liste	Potreban eksplicitni <code>import array</code> modula za deklaraciju niza
Namenjene za kraće sekvence podataka	Namenjene za veće sekvence podataka
Lakša modifikacija (dodavanje, brisanje) podataka	Teža modifikacija (dodavanje, brisanje) jer se vrši pomeranje ostalih elementa
Memorijski zahtevnije	Memorijski kompaktniji i manje zahtevni
Ugneždene liste mogu biti različite dužine	Ugnežđeni nizovi moraju biti iste dužine
Izvršavaju direktne operacije kroz funkcije: <code>count()</code> <code>sort()</code> <code>max()</code> <code>min()</code> <code>sum()</code> <code>index()</code> <code>append()</code> <code>remove()</code>	Potreban <code>import</code> odgovarajućih modula za izvršavanje različitih operacija

Liste

- Za razliku od niza kod kojeg se sekvencijalna uređenost dobija indeksiranjem elemenata niza, linearni poredak elemenata liste se određuje pokazivačem u svakom elementu na sledeći element u listi
- Svaki pokazivač pokazuje na jedan (sledeći) element liste i ti pokazivači povezuju elemente u jednu celinu tj. u listu
- Pokazivač kao pojam u listama nije isto kao pokazivač u programskim jezicima, on je neka indikacija u jednom elementu liste na to koji je sledeći element u poretку liste

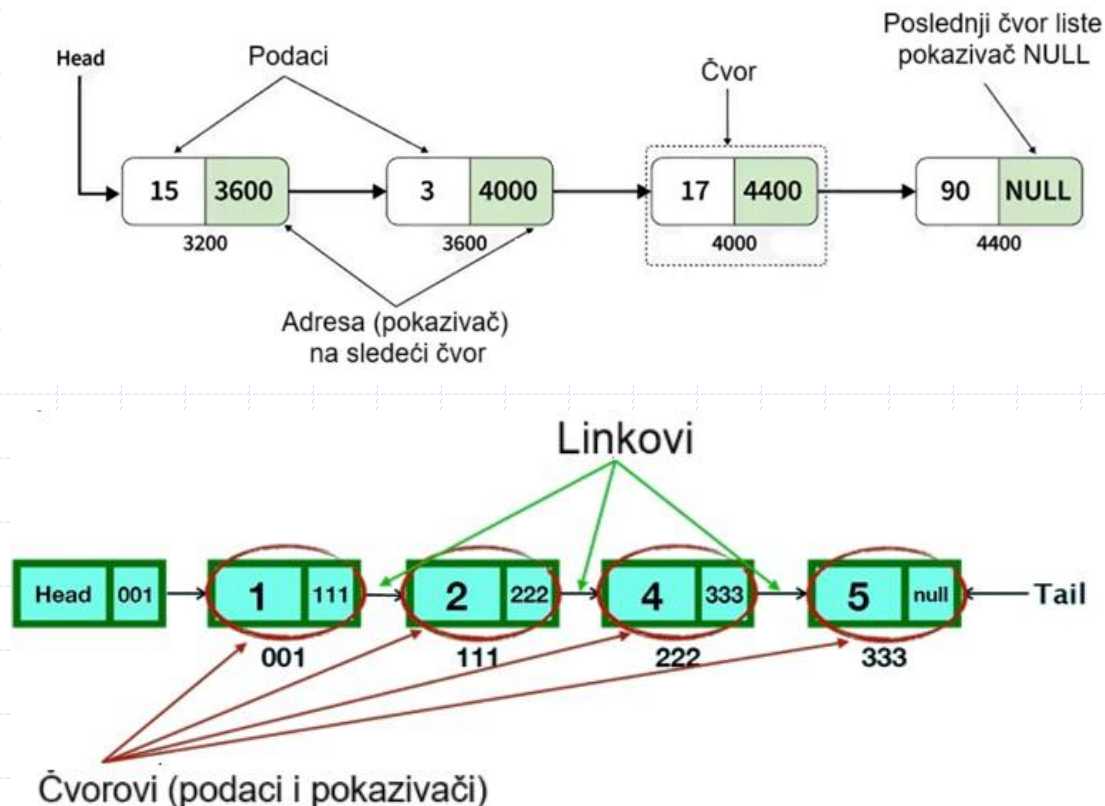


Liste – podela:

- a) Jednostruko povezane liste
- b) Dvostruko povezane liste
- c) Kružne liste

Jednostruko povezane liste

- Jednostruko povezane liste (*Singly Linked List*): svaki čvor se sastoji od dva dela, podataka i pokazivača na sledeći čvor. Ova struktura omogućava dinamičko povezivanje čvorova, formirajući niz nalik lancu

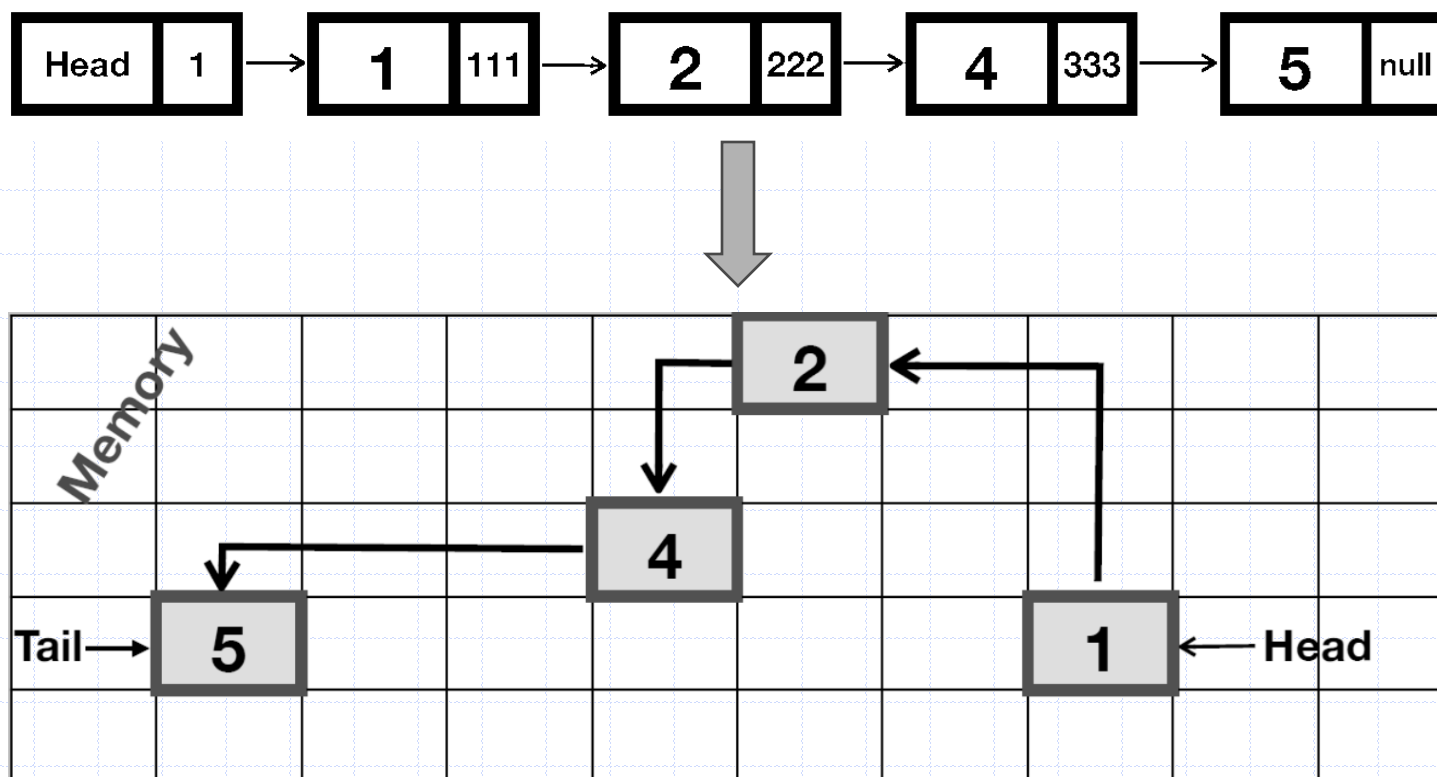


Jednostruko povezane liste

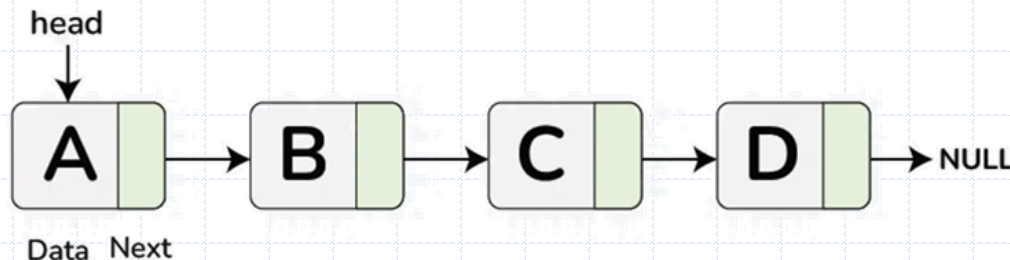
- Broj elemenata liste je veličina ili **dužina liste**
- Lista bez elemenata naziva se **prazna lista**
- Prvi element liste je „glava liste“ (**head**), i nema prethodnika, dok je poslednji element liste „rep“ (**tail**) i nema sledbenika
- Svaki element liste prikazan je objektom sa dva polja koji se naziva **čvor** (*node*), prvo polje čvora je **podatak**, dok je drugo polje **pokazivač** na sledeći čvor liste
- Prvo polje čvora se generalno označava kao ključ i predstavlja pravi sadržaj elementa liste, dok se drugo polje naziva sled i pokazuje na sledbenika elementa liste
- Listi se grupno pristupa preko posebnog spoljašnjeg pokazivača koji nije deo liste i koji pokazuje na prvi čvor liste
- **NULL** je specijalna vrednost pokazivača i nju sadrži poslednji čvor liste
- Ako spoljašnji pokazivač ima vrednost **NULL**, onda je lista prazna

Memorisanje podataka liste

- Memorijski prostor za smeštanje liste nije kontinualan kao kod nizova, tako da se elementi jednostruko povezane liste smeštaju na **različitim lokacijama u memoriji**



Definisanje čvora liste



Definisanje čvora u jednostruko povezanoj listi

```
class Node:
    def __init__(self, data):
        # Deo čvora sa podacima
        self.data = data
        self.next = None
```

Prilikom definisanja metode `__init__(self)` podrazumevani parameter **self** se uvek prosleđuje kao argument i predstavlja objekat same klase, **self.next=None** znači da se kreira inicijalno prazna lista

- Klasa `Node` sadrži celobrojno polje podataka (`data`) za čuvanje informacija i pokazivač na drugi čvor (`next`) za uspostavljanje veze sa sledećim čvorom u listi
- `__init__` je posebna metoda poznata kao konstruktor, automatski se poziva kada se kreira nova instanca (objekat) klase i omogućava inicijalizaciju atributa (promenljivih) objekta
- Pažnja!!! Sintaksa je „dve donje crte bez razmaka ispred i iza reči init“

__init__

Operacije za rad sa listama

Osnovne operacije za rad sa povezanim listama su:

1) Dodavanje novog elementa u listu:

- Dodavanje novog elementa (čvora) na početak liste
- Dodavanje novog elementa (čvora) na kraj liste
- Dodavanje novog elementa (čvora) u sredinu/ili na bilo koju poziciju na listi

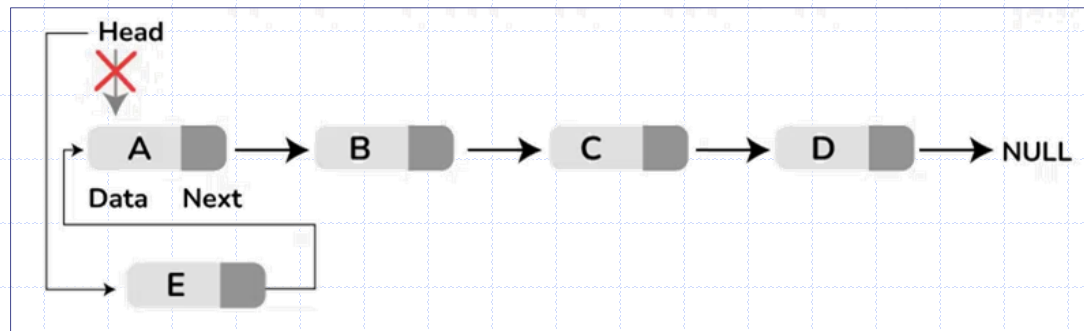
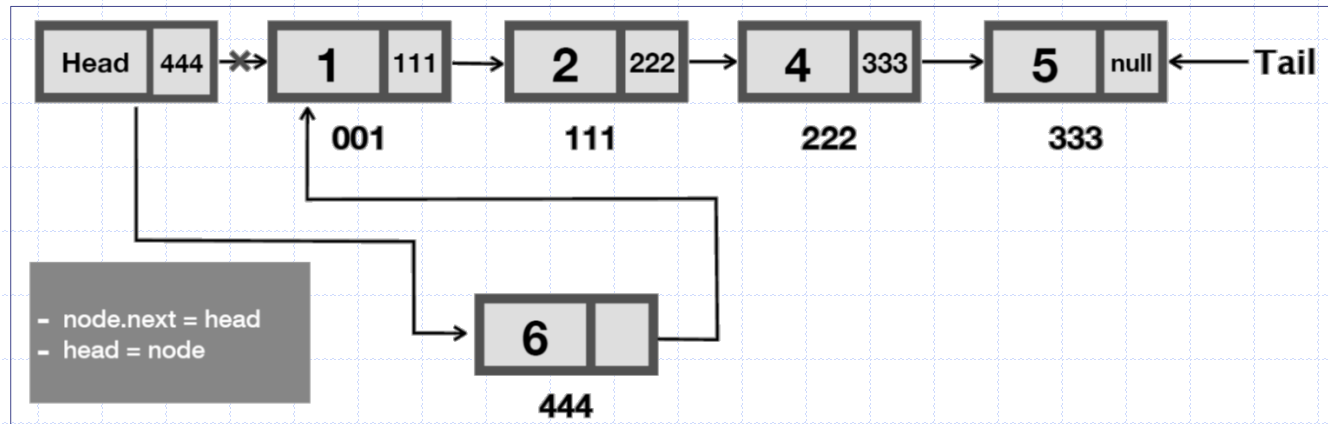
2) Brisanje elemenata liste:

- Brisanje prvog čvora
- Brisanje poslednjeg čvora
- Brisanje čvora u sredini/ili na bilo kojoj poziciji u listi

3) Obilazak (traversal) liste

- > Lista je najpre prazna, zatim se u praznu listu dodaju elementi
- > Pre nego što se element dodaje u listu, potrebno ga je kreirati
- > Kreiranje elementa liste vrši se dinamičkom alokacijom i nakon dinamičke alokacije potrebnog prostora, neophodno je izvršiti inicijalizaciju vrednosti kreiranog čvora

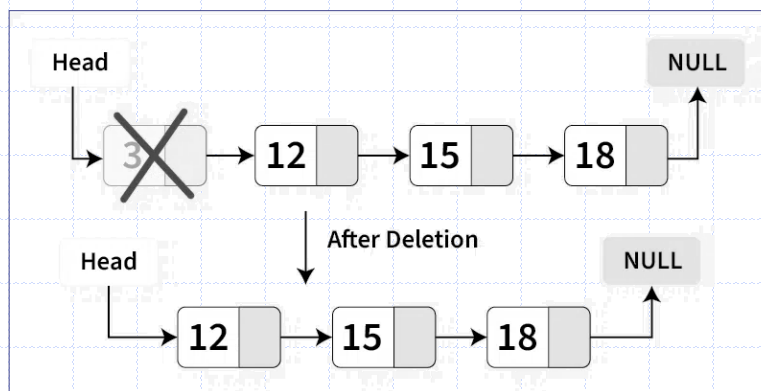
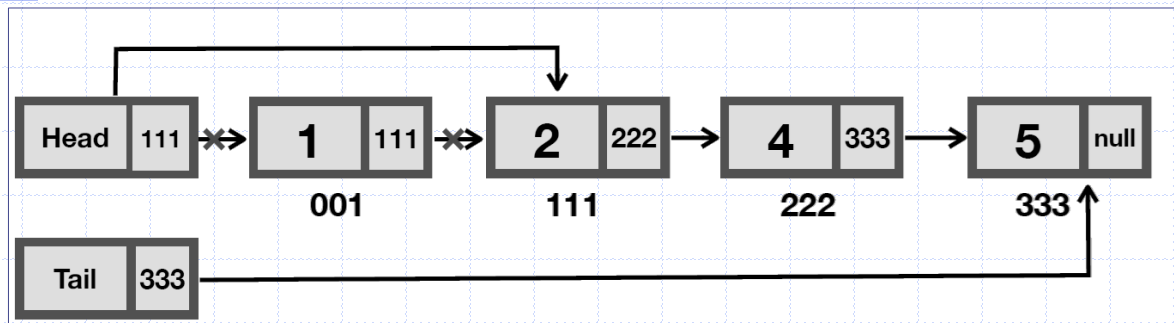
Dodavanje čvora na početak liste



Algoritam:

- Povezati prvi čvor (A) povezan liste sa novim čvorom (E)
- Ukloniti zaglavlje sa originalnog prvog čvora povezan liste
- Novi čvor (E) postaviti kao novo zaglavlje povezan liste

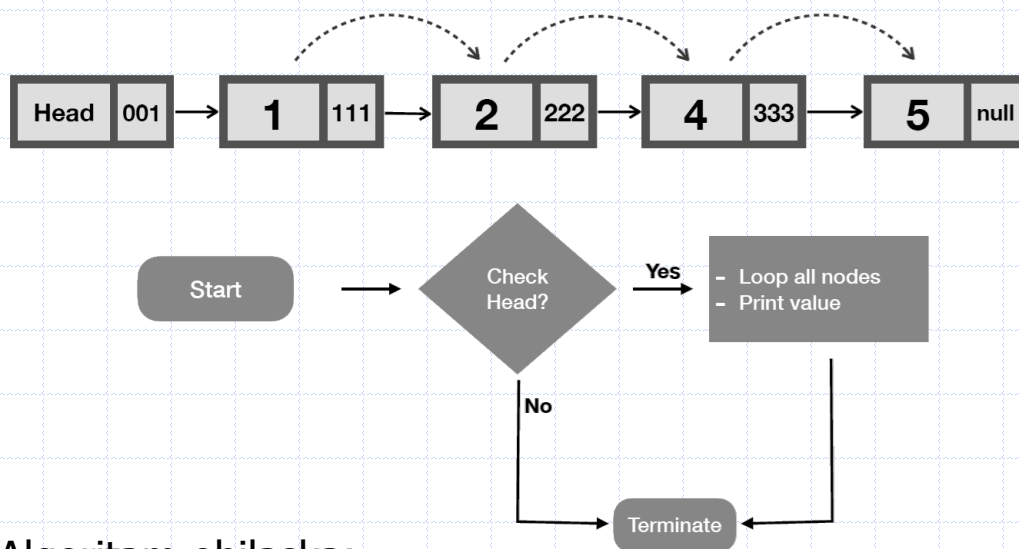
Brisanje prvog elementa liste



Algoritam:

- Ažurirati zaglavlje tako da pokazuje na drugi čvor na listi
- Proveriti da li je zaglavlje **null**, ako jeste (vraća **null**) tada je lista prazna
- Sačuvati **head** tekućeg čvora u privremenoj promenljivoj **temp**, pomeriti pokazivač na sledeći čvor
- Obrisati **temp** čvor i vratiti vrednost novog zaglavlja povezane liste

Obilazak (traversal) liste



```
def traverseList(head):
```

```
#Petlja se izvršava sve dok head  
#ne bude null
```

```
while head is not None:
```

```
# Print podataka čvora
```

```
print(head.data, end=" ")
```

```
# Prelazak na sledeći čvor
```

```
head = head.next
```

```
print()
```

Algoritam obilaska:

- Obilazak jednostruko povezane liste znači da se prolazi kroz svaki čvor liste
- Traversal algoritam može biti [Iterativni](#) ili [Rekurzivni](#)
- [Iterativno](#): Obilazak kroz jednostruko povezanu listu znači ispisivanje vrednosti svakog čvora, prelazak na sledeći čvor dok se ne stigne do poslednjeg čvora čiji pointer ukazuje na **null**
- [Rekurzivno](#): Za obilazak liste koristi se rekurzivna funkcija, počinje se od **heada** prvog čvora, proverava se da li je **null** ili ne i ispisuje se njegova vrednost, a zatim se ponovo poziva funkcija obilaska za sledeći čvor itd...

Liste – primeri metoda

Kreiranje liste:

[] uglaste zagrade

```
a = [1, 2, 3, 4, 5] # List of integers  
b = ['apple', 'banana', 'cherry'] # List of strings  
c = [1, 'hello', 3.14, True] # Mixed data types
```

```
print(a)  
print(b)  
print(c)
```

Output

```
[1, 2, 3, 4, 5]  
['apple', 'banana', 'cherry']  
[1, 'hello', 3.14, True]
```

list() konstruktor

```
a = list((1, 2, 3, 'apple', 4.5))  
print(a)  
  
b = list("GFG")  
print(b)
```

Output

```
[1, 2, 3, 'apple', 4.5]  
['G', 'F', 'G']
```

Metode za liste



LIST

- append()
- copy()
- count()
- insert()
- reverse()
- remove()
- sort()
- pop()
- extend()
- index()
- clear()

Liste – primeri metoda

Pristup elementima liste:

Elementima liste se pristupa indeksiranjem. Indeksi počinju od 0, a[0] je prvi element. Negativni indeksi omogućavaju pristup od kraja (-1 je indeks poslednjeg elementa, -2 preposlednjeg itd)

```
a = [10, 20, 30, 40, 50]
print(a[0])
print(a[-1])
print(a[1:4]) # elementi od indeksa 1 do 3
```

Output

```
10
50
[20, 30, 40]
```

Dodavanje elemenata u listu:

Dodavanje elemenata u listu moguće je izvršiti sledećim metodama: **append()**, **insert()**, **extend()** i **clear()**

```
a = []
a.append(10)
print("After append(10):", a)

a.insert(0, 5)
print("After insert(0, 5):", a)

a.extend([15, 20, 25])
print("After extend([15, 20, 25]):", a)

a.clear()
print("After clear():", a)
```

Output

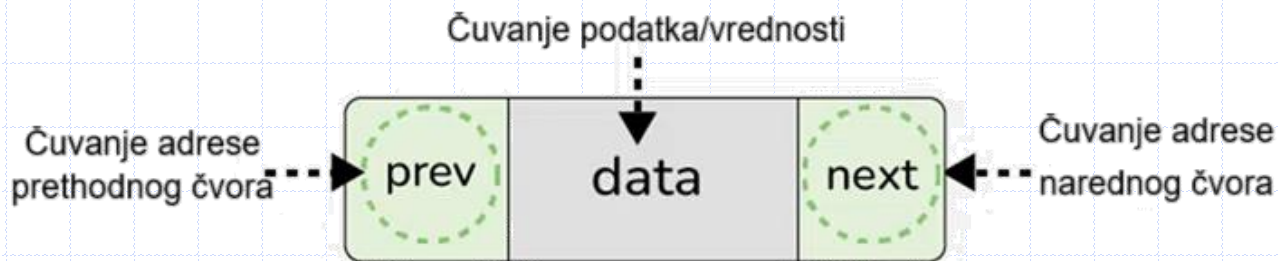
```
After append(10): [10]
After insert(0, 5): [5, 10]
After extend([15, 20, 25]): [5, 10, 15, 20, 25]
After clear(): []
```

Vremenska i prostorna složenost operacija kod jednostruko povezanih lista

	Vremenska složenost	Prostorna složenost
Kreiranje liste	$O(1)$	$O(1)$
Unos novog elementa liste	$O(n)$	$O(1)$
Obilazak liste	$O(n)$	$O(1)$
Brisanje čvora	$O(n)$	$O(1)$
Brisanje liste	$O(1)$	$O(1)$

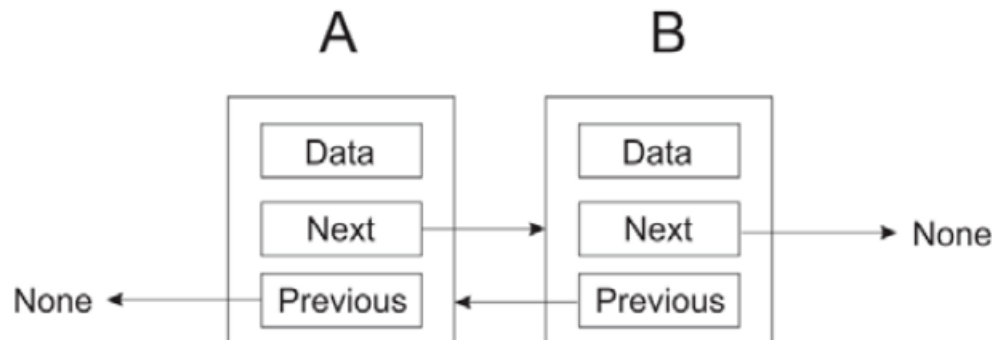
Dvostruko povezane liste

- **Dvostruko povezane liste** (*Doubly Linked List*): Kod njih je omogućeno dvosmerno kretanje kroz listu, jer svaki element mora imati vezu sa prethodnim (**prev**) i idućim elementom (**next**)
- Svaki element liste zauzima **tri adrese**: jednu za podatak (ključ, data), drugu za adresu prethodnog elementa i treću za adresu narednog elementa



Dvostruko povezane liste

- Dvostruko povezana lista omogućava jednostavno kretanje unapred i unazad kroz listu
- Dodavanje i brisanje elemenata iz dvostruko povezane liste obuhvata održavanje vrednosti oba pokazivača
- Funkcije za rad sa dvostruko povezanim listama moraju da vode računa o oba pokazivača i veoma je bitan redosled njihovog postavljanja
- Prvi čvor dvostruko povezane liste u polju prethodnika i poslednji čvor u polju sledbenika imaju vrednost `null`
- Kao i kod jednostruko povezane liste, dvostruko povezanoj listi se pristupa preko spoljašnjeg pokazivača



Kod dvostruko povezane liste čvor A referencira čvor B, ali postoji i referenca unazad, sa čvora B na čvor A
Na slici je primer liste sa dva elementa

